Field Hours

CSCE 647 Image Synthesis

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

A pinhole at (0, 0, 5)

You form an image with a pinhole at (0, 0, 5) and an image plane at z = 4, one unit in front of the pinhole. A scene point and the pinhole determine a line, and that line meets the plane in one place. You store the meeting point as a pixel. A point on the z axis projects along that axis and hits the center of the plane. This projection is the mechanism that assigns scene points to pixels.

You turn the hit into a column using the plane width. The plane runs from x = -1 to x = 1, so a hit at x = 0.5 is halfway from the center to the right edge. On a 4 by 4 image that hit falls on a boundary between two columns. You give each column a half-open interval so the boundary belongs to exactly 1 of the 4 columns.

A scene point, a line through the pinhole, one hit, one column.

Scene point -> Line through pinhole -> Hit on the plane -> One pixel column

The origin through the pinhole

1. The pinhole is at (0, 0, 5). The plane is z = 4 and runs from x = -1 to x = 1.

2. The origin, the pinhole, and the plane center are on one line, so the hit is the center (0, 0, 4).

3. x = 0 is the boundary between column 1 and column 2 of the 4 columns. Your half-open rule assigns it to one column.

Problem 1. The pinhole is at (0, 0, 5) and the plane is at z = 4. A scene point lies on the z axis. Where does it hit in x and y?

Answer. At x = 0 and y = 0, the center of the plane. The line is the z axis.

Problem 2. Why does a boundary between two of the 4 columns need a half-open rule?

Answer. So the boundary point belongs to exactly one column. Without the rule you would count it in two columns or drop it.

Lecture 2 · 90 minutes

A lamp and a cosine

You shade a visible hit with light that comes straight from a lamp. That is local illumination. The lamp is at (0, 4, 0). The hit is at (0, 0, 0) with normal (0, 1, 0), so the direction to the lamp matches the normal and the cosine is 1. You multiply intensity 3 by albedo 0.2 by that cosine and get a diffuse brightness of 0.6. A cosine of 0 would zero the product.

You add a specular term only around the mirror of the lamp direction. If the eye sits on that reflected ray, the

specular weight is 1. If the eye is 30 degrees off it, you raise the cosine of that 30 degree gap to a shininess power. With power 2, a cosine of 0.5 becomes 0.25. You force the specular term to 0 when the diffuse cosine was clamped, so a back face does not glint.

Both terms are local. Only the specular term looks at the eye.

Diffuse term

Lamp direction, normal

Cosine clamped at 0

Intensity times albedo

Specular term

Mirror of the lamp

Raised to a power

Off if diffuse is 0

Brightness 0.6, then a glint

1. Intensity 3, albedo 0.2, and cosine 1. The diffuse term is 3 times 0.2 times 1, which is 0.6.

2. The eye is 30 degrees off the mirror direction, and the cosine of that angle is 0.5.

3. Shininess 2 turns 0.5 into 0.25. You multiply the specular coefficient by 0.25 only if the diffuse cosine was positive.

Problem 1. Intensity is 3, albedo is 0.2, and the cosine is 0. What is the diffuse brightness?

Answer. 0. The product is 3 times 0.2 times 0.

Problem 2. The cosine to the lamp is negative, and the eye sits on the mirror direction. What specular weight do you use?

Answer. 0. You turn the specular term off when the diffuse cosine was clamped to 0.

Lecture 3 · 90 minutes

A segment to the lamp

You test a shadow by walking from the hit toward the lamp and asking whether any surface lies on that segment. The hit is at (0, 0, 0) and the lamp is at (0, 4, 0), so the segment is 4 units long. A triangle that crosses it at distance 1 puts the hit in shadow, and you drop the direct term. You do not drop light the surface itself emits. Emission is not the lamp term.

You bias the test so the hit does not shadow itself. A numerical intersection at distance 0.0001 is the surface you started from. You ignore intersections closer than 0.01. A blocker at distance 1 is farther than the bias and closer than the lamp, so you count it. You pick 0.01 because it is small beside a segment of length 4 and large beside the noise in the hit.

The shadow test is a segment. Hits nearer than the bias are the surface itself.

1. Segment from hit to lamp

2. Ignore hits under 0.01

3. A nearer hit shadows

4. Drop the direct term

A blocker at distance 1

1. The lamp is 4 units above the hit at (0, 0, 0).

2. An intersection at distance 0.0001 is under the bias of 0.01, so you ignore it as self-intersection.

3. An intersection at distance 1 is a blocker, and the direct term becomes 0. A hit at distance 5 is past the lamp and does not count.

Problem 1. The only intersection on the shadow segment is at distance 0.005, and the bias is 0.01. Is the hit in shadow?

Answer. No. 0.005 is closer than the bias, so you treat it as the surface itself and keep the direct term.

Problem 2. A blocker is at distance 1 and the lamp is at distance 4. Is the hit in shadow?

Answer. Yes. The blocker is farther than the bias of 0.01 and closer than the lamp, so you drop the direct term.

Lecture 4 · 90 minutes

Which hit the pixel sees

You decide the visible surface before you shade. Along a camera ray the visible surface is the closest hit with positive distance. Hits at distances 2 and 7 mean you shade the hit at 2. A raster depth buffer makes the same choice: the fragment with the smaller depth wins. You shade the winner only. Shading the loser and then hiding it wastes the shadow test you would have spent on it.

You treat transparency as continued visibility, not as a darker paint. If the closest surface has opacity 0.5, half the pixel still comes from behind it, so you keep the hit at distance 7 as well and composite. You stop after 3 transparent layers. A stack of glass must not demand an unbounded walk. Opaque surfaces still hide everything farther along the ray.

Opacity decides whether distance 7 is ever shaded. The closest hit is still first.

Opaque winner

Keep distance 2

Discard distance 7

Shade the winner only

Opacity 0.5

Keep both hits

Composite front and back

Stop after 3 layers

Two hits along one ray

1. The camera ray hits surfaces at distance 2 and at distance 7.

2. If the near surface is opaque, you shade distance 2 and never look at distance 7.

3. If the near opacity is 0.5, you shade both and composite them. You do not continue past a third transparent layer.

Problem 1. Hits lie at distances 2 and 7, and the near surface is opaque. Which distance do you shade?

Answer. 2. The closest positive hit is the visible surface, and an opaque surface hides the hit at 7.

Problem 2. The first three surfaces along the ray are transparent. Do you search for a fourth?

Answer. No. You stop after 3 transparent layers, even if more glass sits farther along the ray.

Lecture 5 · 90 minutes

Coordinates on a checker

You color a hit by a texture lookup at (u, v), each from 0 to 1. You store (u, v) on the vertices and interpolate them as you interpolate a normal. A 2 by 2 checker splits the unit square at 0.5. A hit at (0.25, 0.25) sits in one square, and (0.75, 0.25) sits in the neighbor. Those two hits can carry different albedos into the same shade.

You filter the lookup when one pixel covers many texels. If the footprint is about 4 texels wide, you read a mip level that already averaged a block, not the single texel at the center. The center texel may be a black square of the checker while the pixel covers half black and half white. The averaged level keeps the checker from sparkling as the camera at (0, 0, 5) moves.

Interpolate the coordinates, choose a mip level, then shade with that albedo.

Interpolate u and v -> Choose a mip level -> Read the albedo -> Then shade

Two hits on a 2 by 2 checker

1. The checker splits at u = 0.5 and at v = 0.5.

2. (u, v) = (0.25, 0.25) reads one square. (0.75, 0.25) reads the square beside it.

3. A pixel whose footprint covers both squares reads a mip level averaged over about 4 texels, not either center texel alone.

Problem 1. A hit has coordinates (0.75, 0.25) on a checker split at 0.5. Which side of the vertical split is it on?

Answer. The right side. u = 0.75 is past 0.5, while v = 0.25 is still below the horizontal split.

Problem 2. A pixel footprint is 4 texels wide. Why do you avoid the single center texel?

Answer. The center can be one checker color while the footprint covers both colors. A mip level that averaged a block matches the footprint.

Lecture 6 · 90 minutes

Eight rays into the hemisphere

You account for light that bounced before it reached the hit. A path starts at the camera, hits the surface, and then continues into the scene, not only toward the lamp. With 1 extra bounce you catch light that reflected once. A red wall can tint the hit even when the lamp cosine is 0, because the light arrived from the wall. A local lamp term misses that tint. The bounce is the difference.

You estimate the bounce with a fixed set of directions. Cast 8 rays from the hit into the hemisphere. If 2 of them see a bright wall and 6 see darkness, you average the 8 results. You do not keep the brightest. Before the average, you multiply each result by the cosine of its direction and by the albedo. Without the cosine, a grazing sample counts as much as one that faces the normal.

Direct light is the lamp. The bounce is an average of eight weighted rays.

1. Shade the direct lamp

2. Cast 8 bounce rays

3. Weight each by cosine

4. Average the eight

Two bright samples out of eight

1. You cast 8 rays. Two return brightness 4 from a red wall, and the other six return 0.

2. For a toy sample, take cosine and albedo both equal to 1, so each bright sample stays 4.

3. The average is 4 plus 4 plus six zeros, divided by 8, which is 1. You add that 1 to the direct term.

Problem 1. You cast 8 rays, all return brightness 2, and cosine and albedo are both 1. What is the average?

Answer. 2. The eight samples sum to 16, and 16 divided by 8 is 2.

Problem 2. The lamp cosine is 0 and a red wall is visible to some bounce rays. Does a local-only shade show the red tint?

Answer. No. Local illumination uses the lamp. The tint requires the bounce rays. With cosine 0 the direct diffuse term is already 0.

Lecture 7 · 90 minutes

Paths you may drop

You write the pixel as a sum of paths, and you drop a path only when you know it carries no light. A shadow ray that hits a blocker drops the direct path. A bounce into a closed black box returns 0, and that branch can stop. You do not drop a path only because it is dim. With albedo 0.5, a second bounce carries 0.5 times 0.5, which is

0.25, and 0.25 is still a contribution you add.

You keep projection, visibility, and transport as three separate mechanisms. The pinhole at (0, 0, 5) chooses the camera ray. The closest-hit test, or the depth buffer, chooses the visible surface so you do not shade a hidden one. The samples choose how light reached that surface. You can render the same hit with 0 bounce rays or with 8. The pixel still names the same visible point.

Change the bounce count without changing which surface the pixel owns.

Projection picks the ray

Visibility picks the hit

Samples sum the paths

A zero path may stop

A factor of 0.25

1. The albedo is 0.5. Light that bounces off this material twice picks up 0.5 times 0.5.

2. That factor is 0.25. You still add the path. Dim is not the same as zero.

3. A ray into a closed black box returns 0, and that branch stops. The pinhole and the closest hit do not change.

Problem 1. Albedo is 0.5 on each of 2 bounces. What factor multiplies that path?

Answer. 0.25. You multiply 0.5 by 0.5, and you still add the path.

Problem 2. You change the bounce count from 0 to 8. Which two decisions stay fixed?

Answer. The projection, which builds the camera ray from the pinhole at (0, 0, 5), and the visibility test, which keeps the closest hit. Only the transport samples change.

Lecture 8 · 90 minutes

Explaining one pixel

You can now project a scene through the pinhole at (0, 0, 5), shade the visible hit with a local lamp term, cut that term when a shadow segment finds a blocker past the bias of 0.01, paint the hit from texture coordinates, and add a bounce estimate from 8 hemisphere rays. You can stop a path that returns 0, and you can keep a path whose factor is only 0.25. That is enough to explain a pixel.

You debug a suspicious pixel by splitting the sum. Confirm the hit distance first, because the wrong winner makes every later term meaningless. Then confirm the shadow segment. Then ask whether the color came from (u, v) or from a bounce. A red tint with the lamp cosine at 0 has to come from the texture or from the 8 rays. You do not move the lamp until those are separated.

Separate the terms before you decide which mechanism made the color.

1. Confirm the visible hit

2. Then the shadow test

3. Then texture or bounce

4. Add the remaining paths

A pixel that looks too red

1. The closest hit is at distance 2, so the surface at distance 7 is hidden and is not the source of the red.

2. The lamp cosine is 0, so the local diffuse term is 0. The shadow test cannot create that red either.

3. Either (u, v) landed in a red texel, or some of the 8 bounce rays hit a red wall. You check those two sources.

Problem 1. Combine visibility and shadowing. The ray hits at distance 2 and at distance 7. The near surface is opaque, and its shadow segment has a blocker at distance 1, past the bias. What direct brightness do you write?

Answer. 0. You shade only the hit at distance 2, and the blocker sets that hit direct term to 0. You do not shade the hit at distance 7.

Problem 2. Combine texturing and the bounce. The lamp cosine is 0, the texture at (u, v) is red, and all 8 bounce rays return 0. Where does the red come from?

Answer. The texture. The local lamp term is 0 and the bounce average is 0, so the remaining color source is the texture lookup.